

# DESARROLLO DE SOFTWARE

## Clase I: Ingeniería de Software

---

*Conceptos fundamentales del desarrollo de software profesional*

2026



# ¿QUÉ ES EL SOFTWARE?

El software es el conjunto de programas, datos e instrucciones que permiten a una computadora realizar tareas. A diferencia del hardware, el software es intangible y puede ser copiado, distribuido y modificado.

## Funcionalidad

Realiza las tareas para las que fue diseñado.

## Confiabilidad

Funciona correctamente bajo condiciones normales.

## Eficiencia

Usa recursos de forma óptima.

## Mantenibilidad

Puede ser modificado y mejorado fácilmente.

## Usabilidad

Se tiene que entender el funcionamiento, sencillo, simple, apb.

# INGENIERÍA DE SOFTWARE

La Ingeniería de Software aplica principios de ingeniería al diseño, desarrollo, prueba y mantenimiento de software. Combina aspectos técnicos, de gestión y metodológicos para crear sistemas de calidad.

## **Análisis**

Entender el problema y definir requisitos

## **Diseño**

Arquitectura y estructura de la solución

## **Implementación**

Codificación del sistema

## **Pruebas**

Verificar que el sistema funciona correctamente

## **Despliegue**

Puesta en producción y entrega al cliente

## **Mantenimiento**

Mejoras, correcciones y actualizaciones

# TIPOS DE SOFTWARE

## Software Estándar

Bajo coste



Licencia por uso



Universal



Disponible nada más  
contratarlo



Menos errores al ser probado  
por muchos usuarios



Facilidad de adquisición



Un mejor diseño y compatibilidad  
con el hardware y multimedia



# V/S

## Software a Medida

Alto coste



Costes de mantenimiento



Personalizado según tus  
necesidades



Necesita de un tiempo de  
implementación



Más errores por no haber sido  
testado por muchos usuarios



Sirve para todo tipo de  
plataformas

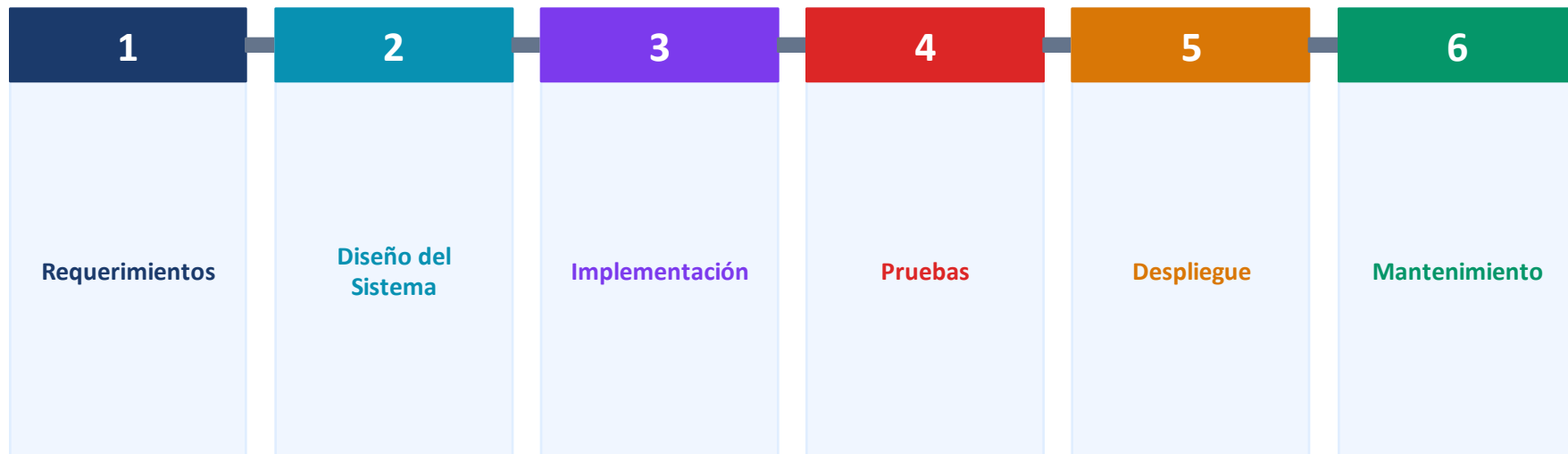


Exclusividad de uso



# CICLO DE VIDA DEL SOFTWARE — SDLC

*El SDLC (Software Development Life Cycle) es el proceso estructurado para planificar, crear, probar y entregar un sistema de software.*



# REQUERIMIENTOS

- Es el proceso crucial de recopilar , estructurar y validar las necesidades de los usuarios y partes interesadas para desarrollar un sistema funcional.
- Objetivo: que debe hacer el software .

# MODELOS DE PROCESO DE SOFTWARE

## Cascada (Waterfall)

Proceso lineal y secuencial. Cada fase debe completarse antes de pasar a la siguiente. Simple pero rígido. Funciona bien cuando los requerimientos son estables.

## Iterativo e Incremental

El sistema se desarrolla en ciclos cortos (iteraciones). Cada iteración agrega funcionalidad. Permite ajustes durante el desarrollo.

## Espiral

Combina el modelo cascada con prototipos. Se centra en el análisis de riesgos. Cada ciclo recorre planificación, análisis de riesgo, ingeniería y evaluación.

## Metodologías Ágiles

Familia de métodos iterativos (Scrum, Kanban, XP). Entregas continuas de valor. Alta colaboración con el cliente. (Ver Clase II)

# ROLES EN UN EQUIPO DE DESARROLLO

## Product Owner / Cliente

Define los requerimientos y prioridades. Representa la visión del negocio.

## Project Manager (PM)

Gestiona el equipo, recursos, tiempos y riesgos del proyecto.

## Analista Funcional

Traduce los requerimientos del cliente a especificaciones técnicas.

## Desarrollador / Dev

Diseña e implementa el código. Puede ser frontend, backend o fullstack.

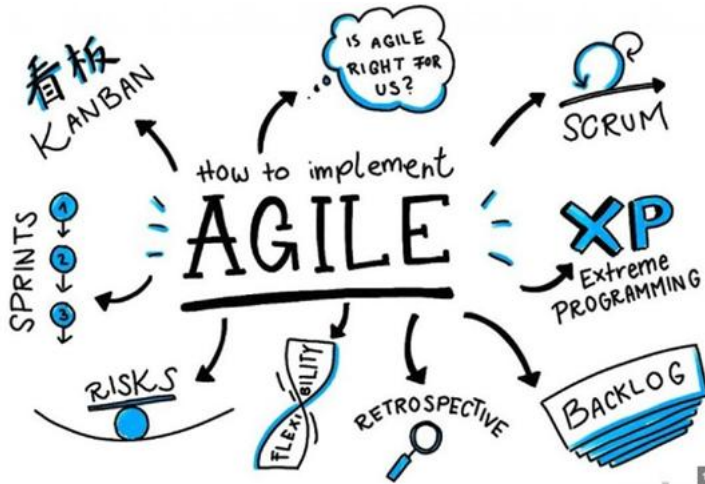
## QA Tester

Verifica la calidad del sistema. Diseña y ejecuta casos de prueba.

## DevOps / Infraestructura

Gestiona deployments, CI/CD, servidores y ambientes de trabajo.

# EL MANIFIESTO ÁGIL



El Manifiesto Ágil es un documento que se centra en los **4 valores y 12 principios** del desarrollo de software con metodologías ágiles. Lo publicaron, en febrero de 2001, 17 desarrolladores de software que necesitaban contar con una alternativa al proceso de desarrollo de productos más lineal y orientado a procesos.

# EL MANIFIESTO ÁGIL — 2001

*4 valores y 12 principios que cambiaron la forma de desarrollar software — firmado por 17 líderes de la industria en 2001.*

## Valoramos MÁS:

✦ **Individuos e interacciones**

✦ **Software funcionando**

✦ **Colaboración con el cliente**

✦ **Respuesta al cambio**

>>

Procesos y herramientas

>>

Documentación exhaustiva

>>

Negociación de contratos

>>

Seguir un plan

## Por encima de:

# SCRUM

## ROLES

### Product Owner

Gestiona el backlog y prioridades

### Scrum Master

Facilita el proceso y elimina impedimentos

### Dev Team

Equipo auto-organizado que construye el producto

## CEREMONIAS

### Sprint Planning

Planificar el sprint (2 sem.)

### Daily Standup

Sincronización diaria (15 min)

### Sprint Review

Demo al cliente al final del sprint

### Retrospectiva

Mejora continua del equipo

# KANBAN Y EXTREME PROGRAMMING (XP)

## KANBAN

- ▶ Tablero visual de estados: Pendiente → En Progreso → Hecho
- ▶ Limita el WIP (Work In Progress) para evitar sobrecarga
- ▶ Flujo continuo: no hay sprints, las tareas fluyen
- ▶ Ideal para equipos de soporte y mantenimiento
- ▶ Herramientas: Jira, Trello, GitHub Projects



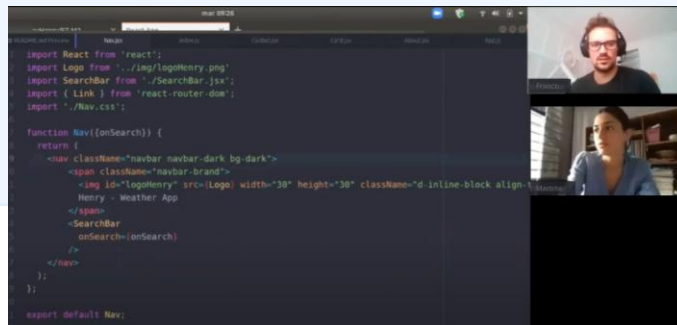
MÉTODO KANBAN

## EXTREME PROGRAMMING (XP)

- ▶ Pair Programming: dos developers, un teclado
- ▶ TDD: Test Driven Development (Desarrollo guiado por pruebas)
- ▶ **Refactoring** continuo del código

Técnica de ingeniería de software orientada a mejorar la calidad interna del código sin alterar su funcionalidad observable.

- ▶ Integración continua (CI)
- ▶ Releases (lanzamientos de software) pequeños y frecuentes:  
Es una práctica fundamental en el desarrollo ágil y DevOps que busca minimizar riesgos, aumentar el valor entregado y obtener retroalimentación rápida



# ÁGIL VS. CASCADA — ¿CUÁNDO USAR CADA UNO?

| Característica       | Cascada (Waterfall)                              | Metodologías Ágiles                          |
|----------------------|--------------------------------------------------|----------------------------------------------|
| Requerimientos       | Fijos desde el inicio                            | Pueden cambiar en cualquier momento          |
| Entregas             | Una sola al final                                | Continuas y frecuentes                       |
| Documentación        | Extensa y formal                                 | Justa y necesaria                            |
| Feedback del cliente | Al final del proyecto                            | En cada iteración                            |
| Cambios              | Costosos y difíciles                             | Bienvenidos y esperados                      |
| Ideal para           | Proyectos de alcance fijo (ej: ingeniería civil) | Productos de software con evolución continua |

# ¿QUÉ SON LOS REQUERIMIENTOS?

Los requerimientos de software son las **necesidades** y **expectativas** que deben cumplirse para que un software sea considerado exitoso. Estos requerimientos pueden provenir de diferentes partes interesadas, como **clientes, usuarios, desarrolladores y gerentes**.

Un requerimiento describe lo que un sistema debe hacer (funcional) o cómo debe comportarse (no funcional). Capturar bien los requerimientos es la base de un proyecto exitoso.

## Requerimientos Funcionales

Describen las funciones específicas del sistema. Ej: 'El usuario debe poder registrarse con email y contraseña'.

## Requerimientos No Funcionales

Atributos de calidad: rendimiento, seguridad, usabilidad, escalabilidad. Ej: 'El sistema debe responder en menos de 2 segundos'.

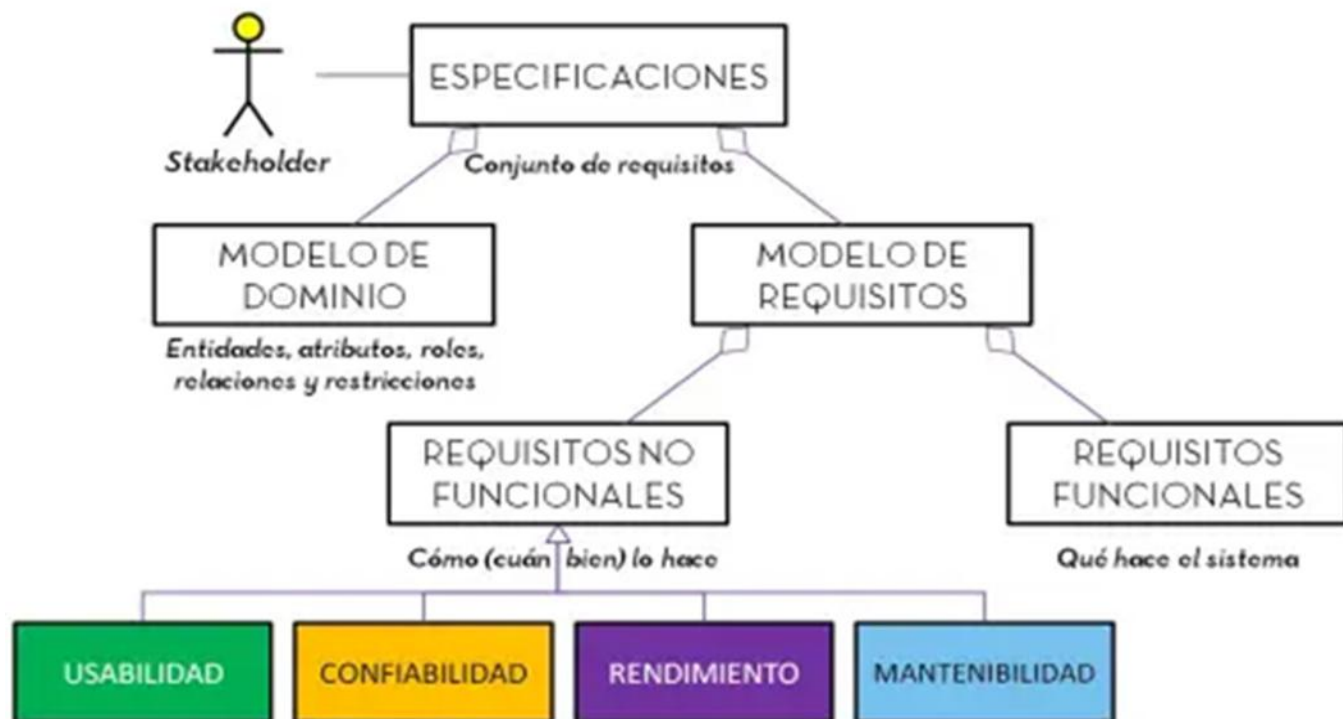
## Requerimientos de Negocio

Describen el "porqué", el objetivo de alto nivel que el sistema debe alcanzar para la organización. Alcances y limitaciones del proyecto. La necesidad comercial ej: aumentar las ventas 10%

## Requerimientos de Usuario

Describen las tareas que el usuario debe poder realizar. Se expresan en lenguaje natural o historias de usuario.

## ANÁLISIS DE REQUISITOS (Software Requirements Analysis)



# IMPORTANCIA DE LOS REQUERIMIENTOS



**Asegurar que el software cumpla con su propósito:** Al definir claramente los requerimientos desde el inicio, se evita la ambigüedad y se reduce el riesgo de desarrollar un software que no satisfaga las necesidades de los usuarios.

**Mejorar la comunicación entre las partes interesadas:** Un proceso de gestión de requerimientos bien definido fomenta la comunicación efectiva entre todas las partes involucradas en el proyecto, lo que ayuda a evitar malentendidos y conflictos.

**Optimizar el uso de recursos:** Al tener una comprensión clara de los requerimientos, se pueden planificar y asignar los recursos de manera más eficiente.

**Facilitar el mantenimiento del software:** Un buen conjunto de requerimientos documentados facilita la comprensión del código y su posterior mantenimiento.



La solicitud del usuario



Lo que entendió el líder del proyecto



El diseño del analista de sistemas



El enfoque del programador



La recomendación del consultor externo



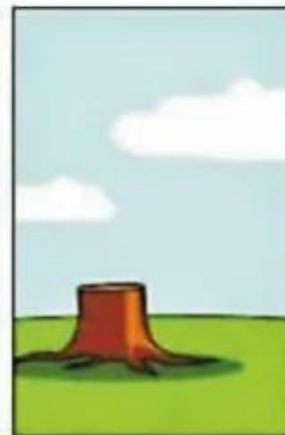
La documentación del proyecto



La implantación en producción



El presupuesto del proyecto



El soporte operativo



Lo que el usuario realmente necesitaba

# HISTORIAS DE USUARIO

Las historias de usuario expresan un requerimiento desde la perspectiva del usuario. Son el corazón de las metodologías ágiles.

## TEMPLATE:

Como **[tipo de usuario]**,  
quiero  
**[acción/funcionalidad]**,  
para  
**[beneficio/objetivo]**.

## EJEMPLO:

Como cliente del e-commerce, quiero filtrar productos por categoría, para encontrar lo que busco más rápido.

## CRITERIOS DE ACEPTACIÓN — Definen cuándo una historia está 'terminada':

- ✓ El filtro muestra solo productos de la categoría seleccionada.
- ✓ Se puede aplicar más de un filtro simultáneamente.
- ✓ El resultado aparece en menos de 1 segundo.

# TÉCNICAS DE RELEVAMIENTO DE REQUERIMIENTOS



## Entrevistas

Reuniones con stakeholders para recopilar necesidades. Puede ser estructurada (preguntas fijas) o no estructurada (conversación abierta).



## Workshops / Talleres

Sesiones grupales con usuarios y técnicos. Permiten resolver conflictos y alinear expectativas rápidamente.



## Observación

El analista observa al usuario trabajar en su entorno real. Revela necesidades implícitas que el usuario no sabe articular.



## Prototipado

Crear maquetas o prototipos de baja/alta fidelidad para validar con el usuario antes de desarrollar.



## Cuestionarios

Útiles para recopilar información de muchos usuarios simultáneamente. Eficientes pero con menor profundidad.



## Análisis de documentos

Revisión de manuales, formularios y sistemas existentes para entender el contexto actual.

# PARADIGMAS DE PROGRAMACIÓN

## Imperativo / Procedural

Se indica CÓMO hacer algo paso a paso.

El código es una secuencia de instrucciones. Ej: C, Pascal.

## Declarativo

Se indica QUÉ se quiere lograr, no cómo

Ej: SQL ('dame todos los clientes con deuda > 0').

## Funcional

Trata el cómputo como evaluación de funciones matemáticas. Sin efectos secundarios, inmutabilidad. Ej: Haskell, Elixir, partes de JS.

## Orientado a Objetos (OOP)

Modela el mundo como objetos con atributos y comportamientos. Principios: encapsulamiento, herencia, polimorfismo. Ej: Java, Python, C#.

# PARADIGMAS DE PROGRAMACIÓN

Existen diferentes maneras de organizar el código y estructurar los programas.

**Los paradigmas de programación más comunes son:**

se basa en la definición de funciones como bloques de código reutilizables.

**FUNCTIONAL  
PROGRAMMING (FP)**

Haskell

03

Se basa en la creación de objetos que encapsulan datos y comportamiento.

**OBJECT-ORIENTED  
PROGRAMMING**

Java, C#

02

**STRUCTURED  
PROGRAMMING**

C, Pascal

01

Se basa en la descomposición de problemas en subproblemas más pequeños y manejables.

# POO – PROGRAMACIÓN ORIENTADA A OBJETOS

**POO se basa en la creación de objetos que encapsulan datos y comportamiento.**

Las principales características de la POO son encapsulamiento, abstracción, herencia, polimorfismo, clases y objetos, y relaciones entre objetos.

## Ventajas de utilizar POO

- **Reutilización de código:** Puedes crear y reutilizar clases y objetos en diferentes partes de tu aplicación, evitando la duplicación de código y mejorando la eficiencia.
- **Modularidad:** Al dividir tu aplicación en módulos o clases independientes, puedes mejorar la organización y facilitar el mantenimiento y la actualización del código.
- **Flexibilidad y escalabilidad:** La POO te permite extender y modificar fácilmente el comportamiento de tus objetos a medida que evolucionan los requisitos de tu aplicación.
- **Mantenibilidad:** La estructura y la organización de la POO hacen que el código sea



# CARACTERÍSTICAS DE POO

## **Encapsulamiento**

Agrupar datos (atributos) y el comportamiento asociado a ellos (métodos) dentro de una unidad llamada objeto.  
Oculta la implementación interna de los objetos, exponiendo solo una interfaz pública (métodos) para interactuar con ellos.  
Promueve la modularidad y la reutilización del código.

## **Abstracción**

Representa las características esenciales de un objeto, ignorando detalles irrelevantes.  
Permite crear modelos conceptuales más simples y fáciles de entender.  
Facilita la creación de jerarquías de clases y la herencia.

## **Herencia**

Permite a una clase heredar atributos y métodos de otra clase padre.  
Promueve la reutilización del código y la especialización.  
Facilita la creación de jerarquías de clases y la organización del código.

## **Polimorfismo**

Permite que objetos de diferentes clases respondan al mismo mensaje de manera diferente.  
Se implementa a través de:

- Sobrecarga de métodos: permite definir métodos con el mismo nombre pero diferentes parámetros o tipos de retorno.
- Sobrescritura de métodos: permite redefinir un método heredado de una clase padre en una clase hija.

# PROGRAMACIÓN FUNCIONAL

Expresamos nuestra lógica sin escribir controles de flujo (Ciclos o Condicionales)

Es funcional porque delega el control de flujos y condicionales a las funciones

```
1 package com.company;
2
3
4 import java.util.List;
5
6 public class Funcional {
7
8     public static void main(String[] args) {
9         List<Integer> numeros = List.of(78, 1, 4, 2, 67);
10        Long resultado = numeros.stream().filter(num -> num < 10).count();
11        System.out.println("Numeros menores a 10: " + resultado);
12    }
13 }
```

Conceptos

Funciones

# PRINCIPIOS SOLID – EJEMPLOS APLICABLES

S

**(Single Responsibility): Una clase solo debe hacer una cosa.**

Cómo aplicarlo: Si tenés una clase *Factura* que calcula el total y también la guarda en la base de datos, sepárala. Creá una clase *Factura* y otra *FacturaRepository* para el guardado.

O

**(Open/Closed): El código debe estar abierto a extensión pero cerrado a modificación.**

Cómo aplicarlo: Si necesitas agregar un nuevo método de pago (ej. Cripto), no modifiques tu función principal de pagos con mil if/else. Usa una interfaz y agrega una nueva clase para Cripto.

L

**(Liskov Substitution): Una clase hija debe poder usarse en lugar de su clase padre sin romper nada.**

Cómo aplicarlo: Si tenés una clase *Ave* con el método *volar()*, no crees una subclase *Pinguino* que herede de ella, porque los pingüinos no vuelan y romperías la lógica.

I

**(Interface Segregation): Mejor muchas interfaces chiquitas que una gigante.**

Cómo aplicarlo: No obligues a una clase a implementar métodos que no usa. Si tenés una interfaz *AccionesTrabajador*, sepárala en *TrabajadorPresencial* y *TrabajadorRemoto* para que el remoto no tenga que implementar un método *ficarEntradaOficina*.

D

**(Dependency Inversion): Depende de abstracciones (interfaces), no de cosas concretas.**

Cómo aplicarlo: Tu lógica de negocio no debería depender de si usás MySQL o MongoDB. Debería depender de una interfaz *BaseDeDatos*.

# PATRONES DE DISEÑO — CREACIONALES

## ¿QUÉ SON LOS PATRONES DE DISEÑO?

Estos patrones proporcionan un enfoque estructurado y reutilizable para resolver situaciones recurrentes en el desarrollo de software.

Cada patrón de diseño tiene un propósito específico y ofrece una solución a un problema de diseño común.

Utilizar los patrones resulta en un software más eficiente, mantenible y escalable.

Es importante tener en cuenta que los patrones de diseño no son algoritmos o código listo para usar, sino más bien pautas y descripciones de soluciones de diseño.

Los desarrolladores deben adaptar e implementar los patrones de diseño de acuerdo con los requisitos y características específicas de su proyecto de software.

Son técnicas para resolver problemas recurrentes de diseño de software.



### COMPORTAMENTALES

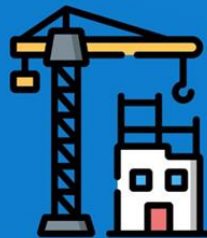
Establece soluciones relacionados con el comportamiento de la aplicación respecto a la interacción entre objetos y clases.



Ej. Observer, State.

### CREACIONALES

Soluciona la creación de objetos, hace un sistema independiente de cómo sus objetos son creados.



Ej. Singleton, Factory.

### ESTRUCTURALES

Describe cómo los objetos se componen para formar estructuras complejas.



Ej. Adapter, Decorator.

# PATRONES DE DISEÑO — CREACIONALES

Los patrones de diseño son soluciones reutilizables a problemas comunes de diseño de software. Propuestos por la 'Gang of Four' en 1994.

## Singleton

Garantiza que una clase tenga solo UNA instancia. Uso: conexión a base de datos, configuración global.

## Factory Method

Define una interfaz para crear objetos, pero deja a las subclases decidir cuál clase instanciar.

## Abstract Factory

Crea familias de objetos relacionados sin especificar sus clases concretas.

## Builder

Construye objetos complejos paso a paso. Separa la construcción de la representación.

## Prototype

Crea nuevos objetos copiando (clonando) un objeto existente.

## Object Pool

Reutiliza objetos costosos de crear. Uso: conexiones de DB, hilos de ejecución.

# PATRONES ESTRUCTURALES Y DE COMPORTAMIENTO

## ESTRUCTURALES

### Adapter

Convierte la interfaz de una clase en otra que el cliente espera.

### Decorator

Agrega responsabilidades a un objeto dinámicamente.

### Facade

Proporciona una interfaz simplificada a un sistema complejo.

### Composite

Compone objetos en estructuras de árbol para representar jerarquías.

### Proxy

Proporciona un sustituto o marcador de posición para otro objeto.

## COMPORTAMIENTO

### Observer

Notifica automáticamente a dependientes cuando un objeto cambia.

### Strategy

Define familia de algoritmos intercambiables en tiempo de ejecución.

### Command

Encapsula una acción como objeto, permitiendo deshacer/rehacer.

### Iterator

Proporciona acceso secuencial a elementos sin exponer la estructura.

### Template Method

Define el esqueleto de un algoritmo, delegando pasos a subclases.

# PATRONES DE DISEÑO — EJEMPLOS APLICABLES

## Creacionales (Cómo crear objetos)

- **Singleton:** Úsalo cuando necesites **una sola instancia** en toda la app (ej. la configuración del sistema o la conexión a la DB).
- EJ: Un sistema de **Log de errores**. No quieres 10 archivos de log abiertos al mismo tiempo; usas una sola instancia global para que toda la app escriba en el mismo lugar.
- **Factory Method:** En lugar de hacer `new Objeto()`, llamas a una "fábrica" que decide qué objeto darte según la situación.
- EJ. Un sistema de **Logins sociales**. Según el botón que toque el usuario (Google, Facebook o GitHub), la "fábrica" te devuelve el objeto de autenticación correcto.

## Estructurales (Cómo organizar clases)

- **Adapter:** Como un adaptador de viaje. Sirve para que dos clases con interfaces incompatibles puedan trabajar juntas.
- EJ. Quieres usar una librería moderna de mapas que usa coordenadas `(lat, long)`, pero tu base de datos vieja guarda los datos en formato `(x, y)`. Creas un **adaptador** que convierta los datos viejos al formato nuevo
- **Decorator:** Para añadir funciones a un objeto sin cambiar su estructura
- EJ. Un sistema de **notificaciones**. Tienes el envío básico por Email, pero puedes "decorarlo" para que también envíe por SMS, y luego otro "decorador" para WhatsApp, sin cambiar la clase original.

## Comportamiento (Cómo se comunican)

**Observer:** Muy común en interfaces visuales. Un objeto avisa a otros cuando cambia algo (ej. cuando llega un mensaje nuevo, se actualiza el icono de notificación). EJ. El sistema de **carrito de compras**. Cuando el stock de un producto llega a cero, el objeto "Producto" le avisa a todos los usuarios que lo tenían en su "Lista de deseos" para que vean el cartel de "Sin stock".

**Strategy:** Te permite cambiar el algoritmo que usás en tiempo de ejecución (ej. elegir entre pagar con Tarjeta o PayPal según lo que elija el usuario).

EJ. Una aplicación de **mapas (GPS)**. Según si eliges ir "Caminando", "En Auto" o "En Bici", el sistema cambia el algoritmo de cálculo de ruta dinámicamente.

# QA/QC

## QUALITY ASSURANCE

VS

## QUALITY CONTROL

**Enfoque proactivo** para **prevenir defectos** haciendo énfasis en el proceso usado en el desarrollo del producto, garantizando que lo propuesto en el plan de calidad se lleve a cabo.

Es una **actividad transversal** en el ciclo de vida del desarrollo del producto y se lleva a cabo mediante **auditoras de calidad regulares** con el objetivo de identificar oportunamente las desviaciones entre lo planeado y lo realizado.

### ¿Quién es responsable del QA?

Cada miembro del **equipo involucrado en el desarrollo del producto** es responsable. Un ejemplo de actividad de QA en el desarrollo de software es la **verificación de especificaciones técnicas**.



**"QA asegura la calidad de los procesos utilizados para crear productos de calidad"**

**Proceso reactivo puntual**, enfocado en **identificar y corregir** los defectos encontrados en el producto ya terminado antes de ser liberado o entregado al cliente final.

### ¿Quién es responsable del QC?

Usualmente es realizado por un **equipo de trabajo independiente** que se encuentra exclusivamente dedicado a probar el producto para encontrar errores. Un ejemplo de actividad de QC en el desarrollo de software es la **validación funcional**.

**"QC es el proceso de probar la calidad de un producto"**



El término de QA es muy utilizado entre las compañías del sector servicios con una gran presencia en el mundo tecnológico.

Aunque el término "QA" es relativamente nuevo, ya que el mundo de "testing", "automatización de pruebas", etc., encontró salida en el mercado IT a mediados del año 2008, muchísima gente confunde control de calidad (QC) con el término QA (Quality Assurance), que en sus siglas en Inglés, viene a significar GARANTÍA DE CALIDAD.

Los nombres son muy parecidos pero se refieren a procesos totalmente distintos, si bien es verdad que el QA se basa mucho en el feedback que se recibe desde QC, aumentando aún más la confusión.

# QA vs QC — GARANTÍA VS. CONTROL DE CALIDAD

| Característica      | QA (Quality Assurance)                               | QC (Quality Control)                          |
|---------------------|------------------------------------------------------|-----------------------------------------------|
| Foco                | Prevención de defectos (proceso)                     | Detección de defectos (producto)              |
| ¿Cuándo actúa?      | Desde el inicio del proyecto                         | Cuando el producto está terminado             |
| ¿Qué hace?          | Define estándares y parámetros de aceptación         | Controla el comportamiento del producto final |
| ¿Quién trabaja con? | Dev, ingenieros, managers, cliente                   | Con el equipo de QA                           |
| Orientación         | Al proceso                                           | Al producto                                   |
| Ejemplo             | Definir criterios de aceptación antes de desarrollar | Ejecutar pruebas sobre el sistema finalizado  |

# PRUEBAS DE SOFTWARE

Las pruebas de software son cruciales para garantizar la calidad y confiabilidad del software. Ayudan a detectar errores tempranamente, prevenir problemas y mejorar la experiencia del usuario.

Las pruebas de software son un conjunto de actividades y procesos que se realizan para evaluar la calidad de un software.

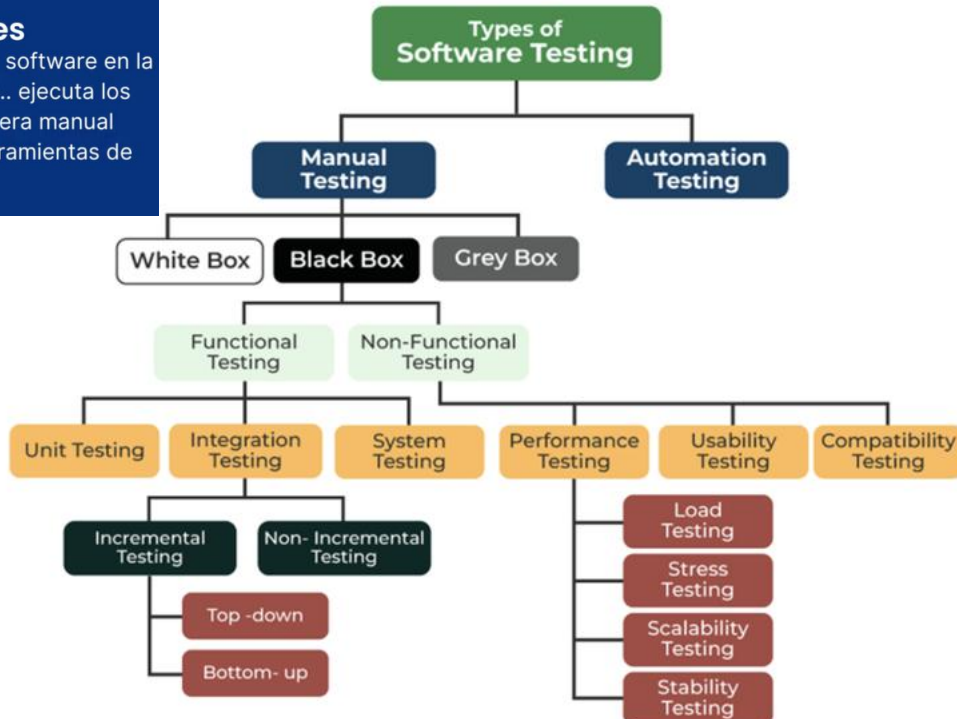
Su objetivo es identificar y corregir errores, verificar que el software cumple con los requisitos especificados y garantizar que funcione de manera correcta y confiable.

Las pruebas de software son una parte integral del desarrollo de software y se realizan en diferentes etapas del ciclo de vida del software.

# TIPOS DE PRUEBAS DE SOFTWARE

## Pruebas Manuales

Son un tipo de prueba de software en la cual el tester/analista/etc.. ejecuta los casos de prueba de manera manual excluyendo el uso de herramientas de automatización.



## Pruebas Automatizadas

Es una técnica de pruebas que consiste en utilizar scripts y herramientas para controlar la ejecución de las pruebas y la comparación de los resultados reales con los resultados esperados.

## Pruebas Automatizadas



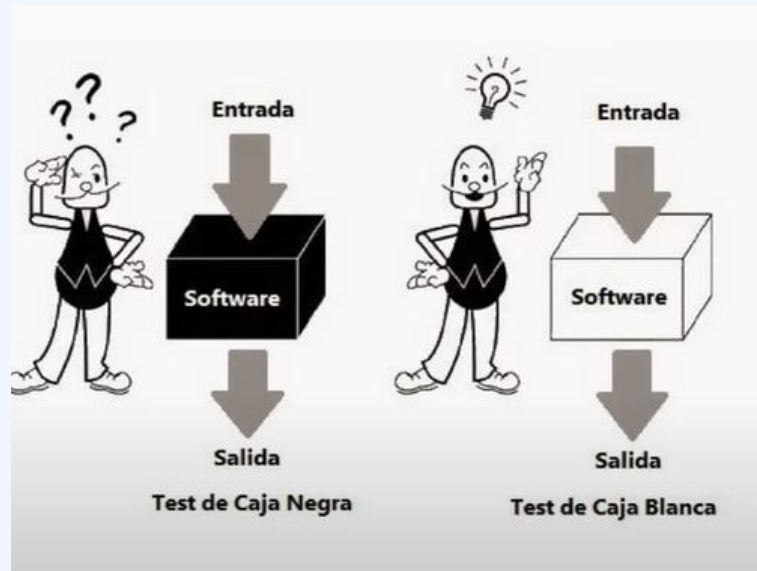
# TIPOS DE PRUEBAS DE SOFTWARE

**Caja Negra (Black-Box):** Se centra en las entradas y salidas, validando los requisitos funcionales desde la perspectiva del usuario final sin conocer la estructura interna.

Es ideal para pruebas funcionales y de sistema.

**Caja Blanca (White-Box):** Requiere conocimientos de programación para analizar el flujo de datos, estructuras lógicas y el código fuente. Es común en pruebas unitarias y de integración para detectar errores ocultos.

**Caja Gris (Gray-Box):** Un enfoque híbrido donde el tester tiene acceso limitado a la arquitectura, base de datos o documentación técnica, permitiendo realizar pruebas más enfocadas que la caja negra.



# TIPOS DE PRUEBAS DE SOFTWARE

## Pruebas Unitarias

Prueban unidades individuales de código (funciones, clases). Aisladas y rápidas. Ej: JUnit, PyTest.

## Pruebas de Integración

Verifican que módulos separados funcionen correctamente juntos. Detectan problemas de interfaz.

## Pruebas de Sistema

Evalúan el sistema completo en su conjunto. Verifican flujos de negocio completos.

## Pruebas de Aceptación (UAT)

El usuario final valida que el sistema cumple sus necesidades reales.  
Determinan si el software cumple con los requisitos definidos por los usuarios o clientes.

## Pruebas de Caja Negra

Sin conocer el código interno. Solo entradas y salidas esperadas. Perspectiva de usuario.

## Pruebas de Caja Blanca

Conociendo el código. Se diseñan casos según rutas de ejecución. Requiere cobertura de sentencias.

# CASO DE PRUEBA

Un caso de prueba es un conjunto de acciones que se realizan en un sistema para determinar si cumple con los requisitos de software y funciona correctamente.

Su propósito es verificar que las diferentes funcionalidades del sistema se comporten como se espera y confirmar que el sistema cumpla con todos los estándares, directrices y requisitos del cliente.

El proceso de escribir un caso de prueba también puede ayudar a detectar errores o defectos en el sistema.

|                                      |                                                                                                                                                                                                                     |
|--------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>ID de Caso de Prueba</b>          | Identificador único del caso de prueba                                                                                                                                                                              |
| <b>Nombre de Caso de prueba</b>      | Caso Uso aspecto a Probar                                                                                                                                                                                           |
| <b>Descripción</b>                   | Se probará la respuesta del sistema cuando se presenta X escenario...                                                                                                                                               |
| <b>Precondiciones</b>                | Condiciones que se deben cumplir para la ejecución del caso de prueba.                                                                                                                                              |
| <b>Relaciones Casos de uso</b>       | Indicar si el caso de uso al cual se le está derivando los casos de prueba tiene conexiones con casos de uso mediante relaciones de include, extend o generalización.                                               |
| <b>Pasos y condiciones ejecución</b> | Pasos detallados del caso de prueba y los datos con los que se probará                                                                                                                                              |
| <b>Resultado Esperado</b>            | Resultado ideal de la aplicación de acuerdo a los pasos ejecutados                                                                                                                                                  |
| <b>Estado Caso de Prueba</b>         | <ul style="list-style-type: none"><li>• Ejecutado<ul style="list-style-type: none"><li>○ Exitoso</li><li>○ Fallido</li><li>○ Frenado</li></ul></li><li>• Pendiente de ejecución</li><li>• En construcción</li></ul> |
| <b>Resultado Obtenido</b>            | Se diligencia luego de la ejecución del caso de prueba y de acuerdo a la reacción de la aplicación                                                                                                                  |
| <b>Errores Asociados</b>             | Al ejecutar el caso de prueba, en este campo se indican los identificadores únicos de los errores presentados.                                                                                                      |
| <b>Responsable Diseño</b>            | Nombre del analista de pruebas que diligenció la plantilla                                                                                                                                                          |
| <b>Responsable Ejecución</b>         | Nombre del tester que ejecutó el caso de prueba                                                                                                                                                                     |
| <b>Comentarios</b>                   |                                                                                                                                                                                                                     |

# HERRAMIENTAS DE TESTING

## JUnit / TestNG

### Unit Testing Java

Frameworks estándar para pruebas unitarias en Java. Integración total con Maven/Gradle y CI/CD.

## PyTest / Unittest

### Unit Testing Python

PyTest es el más popular para Python. Fixtures, parametrización y plugins para cobertura.

## Selenium / Playwright

### Automatización Web

Controlan el browser para simular acciones del usuario. Playwright es la evolución moderna de Selenium.

## Postman / Newman

### Pruebas de API

Postman para pruebas manuales de APIs REST/GraphQL. Newman para ejecutar colecciones en CI/CD.

## SonarQube

### Análisis de Código

Detecta bugs, code smells y vulnerabilidades de seguridad. Muestra porcentaje de cobertura de tests.

## Appium

### Testing Móvil

Automatización de pruebas en apps iOS y Android. Compatible con código nativo, híbrido y web.

# GESTIÓN DE PROYECTOS DE SOFTWARE

La gestión de proyectos de software coordina personas, procesos y herramientas para entregar software de calidad, en tiempo y dentro del presupuesto.

1

## Entregar en tiempo

Cumplir los plazos acordados con el cliente.

2

## Mantener costos

No superar el presupuesto general del proyecto.

3

## Calidad del producto

Entregar software que cumpla las expectativas del cliente.

4

## Equipo óptimo

Mantener un equipo motivado y con buen funcionamiento.

# GESTIÓN DE RIESGOS

La gestión de riesgos identifica y modera las amenazas que pueden afectar el proyecto durante todo su ciclo de vida, antes de que se conviertan en problemas reales.

## 1. Identificación

Listar todos los riesgos posibles del proyecto (técnicos, de personal, externos).

## 2. Análisis

Evaluar probabilidad e impacto de cada riesgo. Priorizar la lista.

## 3. Planeación

Definir planes de mitigación para cada riesgo prioritario.

## 4. Monitorización

Seguimiento continuo durante el desarrollo. Re-evaluar el estado de cada riesgo.

# GESTIÓN DE RIESGOS

| Tipo de riesgo  | Riesgos posibles                                                                                                                                                                                                                               |
|-----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Tecnológico     | La base de datos que se usa en el sistema no puede procesar tantas transacciones por segundo como se esperaba. (1)<br>Los componentes de software de reutilización contienen defectos que hacen que no puedan reutilizarse como se planeó. (2) |
| Personal        | Es imposible reclutar personal con las habilidades requeridas. (3)<br>El personal clave está enfermo e indispuesto en momentos críticos. (4)<br>No está disponible la capacitación requerida para el personal. (5)                             |
| De organización | La organización se reestructura de modo que diferentes administraciones son responsables del proyecto. (6)<br>Problemas financieros de la organización fuerzan reducciones en el presupuesto del proyecto. (7)                                 |
| Herramientas    | El código elaborado por las herramientas de generación de código de software es ineficiente. (8)<br>Las herramientas de software no pueden trabajar en una forma integrada. (9)                                                                |
| Requerimientos  | Se proponen cambios a los requerimientos que demandan mayor trabajo de rediseño. (10)<br>Los clientes no entienden las repercusiones de los cambios a los requerimientos. (11)                                                                 |
| Estimación      | Se subestima el tiempo requerido para desarrollar el software. (12)<br>Se subestima la tasa de reparación de defectos. (13)<br>Se subestima el tamaño del software. (14)                                                                       |

# GESTIÓN DE RIESGOS

| Riesgo                                    | Repercute en        | Descripción                                                                                       |
|-------------------------------------------|---------------------|---------------------------------------------------------------------------------------------------|
| Rotación de personal                      | Proyecto            | Personal experimentado abandonará el proyecto antes de que éste se termine.                       |
| Cambio administrativo                     | Proyecto            | Habrà un cambio de gestión en la organización con diferentes prioridades.                         |
| Indisponibilidad de hardware              | Proyecto            | Hardware, que es esencial para el proyecto, no se entregará a tiempo.                             |
| Cambio de requerimientos                  | Proyecto y producto | Habrà mayor cantidad de cambios a los requerimientos que los anticipados.                         |
| Demoras en la especificación              | Proyecto y producto | Especificaciones de interfaces esenciales no están disponibles a tiempo.                          |
| Subestimación del tamaño                  | Proyecto y producto | Se subestimó el tamaño del sistema.                                                               |
| Bajo rendimiento de las herramientas CASE | Producto            | Las herramientas CASE, que apoyan el proyecto, no se desempeñan como se anticipaba.               |
| Cambio tecnológico                        | Empresa             | La tecnología subyacente sobre la cual se construye el sistema se sustituye con nueva tecnología. |
| Competencia de productos                  | Empresa             | Un producto competitivo se comercializa antes de que el sistema esté completo.                    |

# GESTIÓN DE RIESGOS

| Riesgo                                   | Estrategia                                                                                                                                                                                                                                                      |
|------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Problemas financieros de la organización | Prepare un documento informativo para altos ejecutivos en el que muestre cómo el proyecto realiza una aportación muy importante a las metas de la empresa y presente razones por las que los recortes al presupuesto del proyecto no serían efectivos en costo. |
| Problemas de reclutamiento               | Alerte al cliente de dificultades potenciales y de la posibilidad de demoras; investigue la compra de componentes.                                                                                                                                              |
| Enfermedad del personal                  | Reorganice los equipos de manera que haya más traslape de trabajo y, así, las personas comprendan las labores de los demás.                                                                                                                                     |
| Componentes defectuosos                  | Sustituya los componentes potencialmente defectuosos con la compra de componentes de conocida fiabilidad.                                                                                                                                                       |
| Cambios de requerimientos                | Obtenga información de seguimiento para valorar el efecto de cambiar los requerimientos; maximice la información que se oculta en el diseño.                                                                                                                    |
| Reestructuración de la organización      | Prepare un documento informativo para altos ejecutivos en el que muestre cómo el proyecto realiza una aportación muy importante a las metas de la empresa.                                                                                                      |
| Rendimiento de la base de datos          | Investigue la posibilidad de comprar una base de datos de mayor rendimiento.                                                                                                                                                                                    |
| Subestimación del tiempo de desarrollo   | Investigue los componentes comprados; indague el uso de un generador de programa.                                                                                                                                                                               |

## INFORMES – MÉTRICAS KPI'S

Los KPIs (Key Performance Indicators), también conocidos como Indicadores Clave de Desempeño, son métricas cuantificables que se utilizan para medir el éxito de un proyecto de software o el rendimiento de un equipo de desarrollo

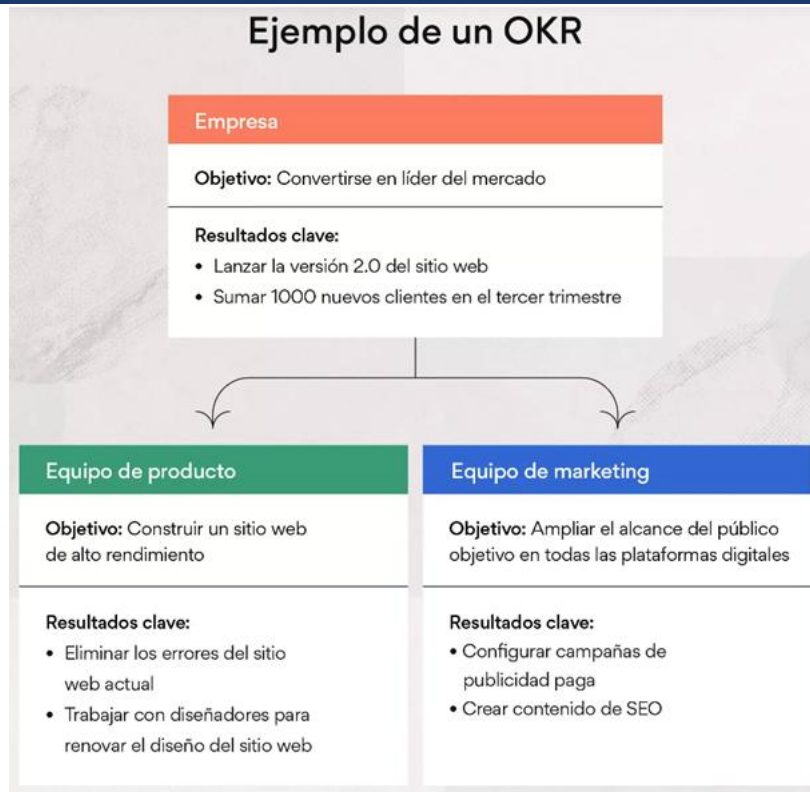


## MÉTRICAS - OKRs

Los objetivos y resultados clave (OKR) combinan los objetivos de negocios con un conjunto de métodos medibles que se implementan para poder alcanzarlos.

Puedes usar los OKR para todo tipo de propósitos, ya sean [objetivos profesionales a largo plazo](#), objetivos trimestrales de equipo u objetivos personales.

### Ejemplo de un OKR



## RESUMEN OKR Y KPI'S

- Los OKR y los KPI son dos métodos de gestión del rendimiento que también pueden ayudarte a alcanzar tus objetivos de diferentes maneras. Los OKR son un marco para la definición de objetivos, mientras que los KPI dan seguimiento al rendimiento de los objetivos.

# ESTILOS ARQUITECTÓNICOS DE SOFTWARE

La arquitectura de software define la estructura de alto nivel de un sistema: sus componentes, sus relaciones y los principios que guían su diseño y evolución.



## Monolítica

Todo el sistema en un único ejecutable. Simple de desarrollar y desplegar, pero difícil de escalar y mantener a gran escala.



## Microservicios

Servicios pequeños e independientes que se comunican por APIs. Escalabilidad y flexibilidad. Ej: Netflix, Amazon.



## Cliente – Servidor

El cliente solicita recursos o servicios al servidor. Modelo clásico de aplicaciones web y de red.



## Serverless

El proveedor cloud gestiona la infraestructura. Solo se paga por ejecución. Ej: AWS Lambda, Azure Functions.



## Event-Driven

Los componentes se comunican mediante eventos. Muy usado en sistemas en tiempo real y microservicios reactivos.



## Hexagonal (Ports & Adapters)

Aísla el núcleo de negocio de los detalles técnicos mediante puertos y adaptadores. Mejora testabilidad.

# COMPARATIVA DE ARQUITECTURAS

| Característica | Monolítica                 | Microservicios             | Cliente-Servidor                                                 |
|----------------|----------------------------|----------------------------|------------------------------------------------------------------|
| Despliegue     | Único artefacto            | Servicios independientes   | Servidor + clientes                                              |
| Escalabilidad  | Vertical (todo)            | Horizontal (por servicio)  | Servidor centralizado                                            |
| Complejidad    | Baja (inicio)              | Alta (orquestación)        | Media                                                            |
| Mantenimiento  | Difícil a gran escala      | Independiente por servicio | Moderado                                                         |
| Resiliencia    | Falla total si falla parte | Aislada por servicio       | Depende del servidor                                             |
| Ejemplos       | Apps legacy, ERP           | Netflix, Amazon, Uber      | Email, Web, FTP( <b>F</b> ile <b>T</b> ransfer <b>P</b> rotocol) |

# HERRAMIENTAS CASE

CASE (Computer-Aided Software Engineering): herramientas que automatizan o asisten en las distintas etapas del desarrollo de software.

## Modelado y Diseño

UMLet, StarUML, draw.io — crean diagramas UML, entidad-relación, casos de uso, flujos y arquitecturas.

## Generación de Código

GitHub Copilot, Tabnine — asisten en la escritura de código. JHipster y Swagger generan código desde modelos.

## Análisis de Calidad

SonarQube, ESLint, Checkstyle — detectan code smells, bugs, vulnerabilidades y código duplicado.

## Gestión de Proyectos

Jira, Trello, Azure DevOps — gestionan tareas, sprints, backlog y seguimiento del equipo.

## Control de Versiones

Git + GitHub/GitLab/Bitbucket — rastrean cambios, permiten colaboración y revisión de código.

## CI/CD y Deployments

Jenkins, GitHub Actions, Docker — automatizan integración continua, pruebas y despliegue a producción.

# HERRAMIENTAS DEL ECOSISTEMA MODERNO

## DISEÑO Y MODELADO

### UMlet / draw.io

Diagramas UML gratuitos. Ideal para modelar antes de codificar.

### StarUML

Soporta UML 2.x completo. Genera código básico desde diagramas.

### Figma / Balsamiq

Prototipado de UI/UX de alta y baja fidelidad.

### Lucidchart

Diagramas colaborativos en línea. Integración con Jira y Confluence.

### PlantUML

Diagramas desde texto plano. Ideal para integrar con docs y código.

## CI/CD Y CALIDAD DE CÓDIGO

### GitHub Actions

Automatiza build, tests y deploys desde repositorio. Gratuito para proyectos públicos.

### Jenkins

Servidor CI/CD open source. Muy configurable, pipeline as code.

### Docker

Conteneriza aplicaciones para deployments reproducibles en cualquier entorno.

### SonarQube

Analiza calidad del código: bugs, vulnerabilidades, deuda técnica y cobertura.

### Postman / Insomnia

Testing de APIs REST. Permite crear colecciones de pruebas automatizadas.

# ¡MUCHAS GRACIAS!

*Clase I: Ingeniería de Software y Desarrollo de Software*

---

"No tengo miedo de un ejército de leones dirigido por una oveja; tengo miedo de un ejército de ovejas liderado por un león"